SensitiveAI: An Ablation Study of Optimization Recipes for Vietnamese Hate Speech Detection

Pham Kha Vy[1] and Cao Hong Nga[1*]

FPT University, Can Tho, Vietnam
phamkhavycm2004@gmail.com
caohongnga@gmail.com
*is the correspondent author.

Abstract. Hate speech on Vietnamese social media is a pressing moderation problem, and fine-tuned multilingual transformers such as mDeBERTa-v3 are now a common approach. Reported gains, however, usually bundle several independent design decisions—head, learning-rate schedule, loss, class weighting, curriculum and decision rule—into one number, so it is unclear which decision matters. We present *SensitiveAI*, a controlled ablation of six systems sharing one fixed split, label mapping, backbone (`microsoft/mdeberta-v3-base`) and seed, differing only in the optimisation recipe, each scored once on the ViHSD test set (three classes HATE / OFFENSIVE / CLEAN, 83% CLEAN). All six land in a narrow 0.654–0.664 macro-F1 and 0.863–0.878 accuracy band; no pairwise macro-F1 difference is significant under a paired bootstrap, and the only significant paired comparisons are in accuracy, where the focal-only probe buys the highest minority recall by giving up CLEAN recall and so loses accuracy. A multi-view head with discriminative learning rates reaches the best macro-F1 (0.6638), a focal-only continuation of the baseline gives the best HATE F1 (0.609) and macro ROC-AUC (0.893), and the highest accuracy (0.878) belongs to the system with the *lowest* macro-F1—the classic imbalanced-classification trap. A three-stage curriculum whose later stages never won validation selection, and a validation-searched threshold rule, did not improve over simpler recipes. We report the full protocol, bootstrap confidence intervals, a confusion-matrix and error analysis, and the negative results, so the comparison can be reproduced rather than re-argued.

Keywords: hate speech detection · Vietnamese NLP · mDeBERTa · class imbalance · ablation study · reproducibility.

1 Introduction

Vietnam has one of the fastest-growing social-media user bases in Southeast Asia, and the volume of Vietnamese text platforms must moderate keeps rising. Flagging hateful and offensive comments automatically is therefore a practical prerequisite for healthier communities, yet Vietnamese remains low-resource and its social-media style is hard (code-switching, "teencode," informal spellings, evasive substitutions, dropped diacritics).

The release of *UIT-ViHSD* [11], a human-annotated corpus of roughly 33k Vietnamese comments labelled CLEAN, OFFENSIVE or HATE, made supervised transformer fine-tuning a realistic default and a standard benchmark for monolingual encoders such as PhoBERT [13] and multilingual ones such as mBERT and XLM-R [2,16,7]. Multilingual DeBERTa-v3 variants have become popular because they combine a large multilingual pre-training corpus with disentangled attention [6,5].

In parallel, many *optimisation* tricks are routinely applied to imbalanced text classification: a richer classification head, discriminative (layer-wise) learning rates, multi-stage fine-tuning, class-weighted or focal losses [9], curriculum learning [1], and per-class decision thresholds [18]. In the literature these are usually bundled together and reported as a single end-to-end improvement, which leaves a practical question unanswered: *which of these decisions actually moves the number on Vietnamese hate speech detection, and by how much?*

This paper answers that with a *controlled ablation*: six systems sharing one backbone (`microsoft/mdeberta-v3-base`), one fingerprinted split, one frozen label mapping, one tokenizer configuration and one seed, differing *only* in the optimisation recipe and each scored once on the held-out test set. The chain (Sect. 4) runs from a plain baseline, to a multi-view pooling head, to a variant adding discriminative rates and class-weighted two-stage training, to a three-stage curriculum.

Our contributions are:

- A comparison of six optimisation recipes for three-class Vietnamese hate speech detection under one fixed split and seed, reported with split fingerprints, per-class metrics, bootstrap intervals and paired significance tests so it can be reproduced rather than re-argued.
- A multi-view classification head (first-token, masked-mean and masked-max pooling) attaining the best macro-F1 at essentially the same parameter count as the standard head.
- An empirical demonstration that the recipes are not uniformly better: no pairwise macro-F1 difference is significant (a failure to detect, not proof of equivalence), the most accurate system has one of the lowest macro-F1 scores, a three-stage curriculum never wins validation selection, and a validation-searched threshold collapses to the trivial 0.5—negative results we discuss for applied moderation.

2 Related Work

2.1 Vietnamese Hate Speech Detection

Hate speech detection is well studied for high-resource languages, but Vietnamese resources appeared comparatively late. The *UIT-ViHSD* corpus [11] is the reference three-class dataset for Vietnamese social media; the VLSP 2019 shared task [21] was an earlier benchmark, and related resources include UIT-ViCTSD [14] and ViHOS for hate/offensive span detection [7]; the SMTCE benchmark

[16] supplies the published numbers we reuse. Vietnamese-specific encoders have since raised the state of the art: ViDeBERTa [19] is a monolingual DeBERTa pre-trained on Vietnamese, ViSoBERT [15] adapts to social-media text and is, to our knowledge, the strongest reported ViHSD encoder, ViHateT5 [17] reformulates the task as text-to-text, and pipelines such as PhoBERT-CNN [20] report higher scores with steps outside our controlled setting.

2.2 Pre-trained Encoders

Fine-tuning pre-trained transformers is the dominant approach, with PhoBERT [13] for Vietnamese and mBERT [4] and XLM-R [2] for multilingual settings. DeBERTa [6] introduced disentangled attention and DeBERTaV3 [5] replaced masked-language modelling with replaced-token detection; the multilingual checkpoint `microsoft/mdeberta-v3-base`, extending this to 100+ languages, is the backbone used here.

2.3 Learning under Class Imbalance

ViHSD is strongly imbalanced and the minority classes are the main error source. Common remedies re-weight the loss by training frequency—the effective number of samples [3] or the inverse frequency—use focal loss, which down-weights easy examples [9], or move the decision threshold away from 0.5. Curriculum learning [1] schedules difficulty or label focus over time, and discriminative fine-tuning with layer-wise rates was popularised by ULMFiT [8]. We fix the encoder and optimiser family and vary exactly these factors; most prior ViHSD work reports a single aggregate improvement, so we deliberately surface the negative findings that single-number reports hide.

3 Dataset

3.1 UIT-ViHSD

We use *UIT-ViHSD* [11], a human-annotated corpus of Vietnamese social-media comments, each carrying exactly one label: CLEAN, OFFENSIVE (insulting but not hate speech) or HATE (targeting a person or protected group).

We follow the official train/dev/test partition. Preprocessing is deliberately light and label-preserving: strip leading/trailing whitespace, collapse runs of whitespace, drop control characters, and drop only rows with no usable text (2 rows in train). We *keep* duplicates and do not lowercase, strip diacritics, normalise teencode or truncate emoji, because those surface cues are part of the signal. This yields 33,398 comments (24,046 / 2,672 / 6,680 for train/dev/test); the corpus has 33,400 rows—the difference is exactly the 2 empty-text train rows (Table 1). Comments are short and uneven (test mean 47.5 characters, median 33; HATE about twice as long as the others). The annotation study reports only moderate inter-annotator agreement ($\kappa \approx 0.5$), which sets a ceiling on accuracy: some test comments are genuinely ambiguous between OFFENSIVE and HATE, and no recipe removes that label noise.

Table 1. The fixed split stored once and reread by every system. Counts are comments after cleaning (Sect. 3). The SHA-256 digest (first 12 hex digits of the full value in the released manifest) is copied into each run directory and re-checked before training.

Split	Comments	HATE	OFF.	CLEAN	SHA-256
train	24,046	2,556	1,605	19,885	`f705ffe94d41`
validation	2,672	270	212	2,190	`6aa552aafb92`
test	6,680	688	444	5,548	`d8f9ae7ddd8c`

3.2 Label Mapping and Imbalance

The native ViHSD identifiers are (0 = CLEAN, 1 = OFFENSIVE, 2 = HATE); because the minority classes are the object of study, we freeze a canonical mapping with the rare class first (HATE = 0, OFFENSIVE = 1, CLEAN = 2), applied once and never changed.

The distribution is skewed and stable across splits: CLEAN is 82–83% of every split, HATE about 10%, and OFFENSIVE the smallest class at 6.6–7.9% (test: 688 HATE, 444 OFFENSIVE, 5,548 CLEAN). Two consequences follow. First, accuracy is misleading—a degenerate CLEAN-always classifier would score about 0.83 yet be useless—so macro-F1 is primary. Second, offensive and hate speech share most surface markers, so the interesting comparison is the trade-off between minority-class F1 and the easy CLEAN class.

3.3 Fixed Split and Reproducibility

The split must be identical for every system, so we store it as CSV files and record each file's SHA-256 digest in a manifest copied into every run directory; all six systems read the same three files and label mapping, leaving the recipe as the only varying factor, with the test split scored once per system after hyperparameters are frozen on validation (Table 1).

3.4 Duplication Across Splits

Because we keep duplicates, we measured their cross-split prevalence. The official ViHSD partition reuses comments: after case-folding, 895 of the 6,680 test comments (13.4%) have an exact duplicate in train or validation and 118 duplicated strings carry conflicting labels, a property of the public split that can only flatter absolute scores. Table 2 re-scores all six systems on the 5,785 duplicate-free comments: accuracy drops by 0.3–0.5 points for every system while macro-F1 and its ordering barely move (M3 still highest at 0.6654; only the two lowest systems swap, by 0.0003), so the conclusions do not rest on the overlap (Sect. 7.2).

4 Methodology

4.1 Backbone and Common Configuration

All six systems use the same encoder, `microsoft/mdeberta-v3-base` [5,12], a multilingual DeBERTa-v3 model with a hidden size of 768 and 12 transformer

Table 2. Full vs. the 5,785 duplicate-free test comments (Sect. 3). Accuracy drops for every system while macro-F1 and its ordering are essentially unchanged, so the conclusions do not rest on the overlap (Sect. 7.2).

	Accuracy		Macro-F1	
System	Full	Dup-free	Full	Dup-free
M1	0.8734	0.8681	0.6545	0.6533
M2	0.8762	0.8719	0.6575	0.6576
M3	0.8754	0.8721	**0.6638**	**0.6654**
M4	0.8731	0.8698	0.6590	0.6601
M5	**0.8783**	**0.8748**	0.6536	0.6537
M6	0.8633	0.8588	0.6604	0.6606

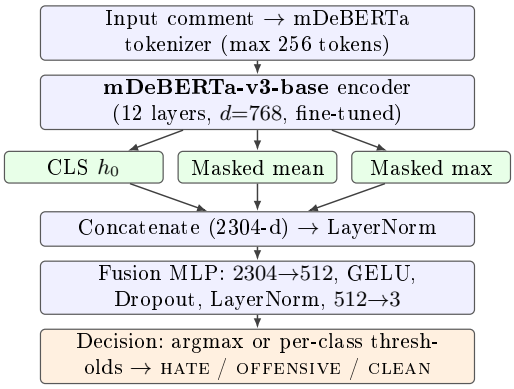


Fig. 1. The SensitiveAI pipeline. All six systems share the encoder and the decision stage; M2–M5 add the multi-view pooling head shown here, whereas M1 and M6 keep the standard first-token head. The recipes ablated on top are the baseline, the multi-view head plus DLR and two-stage training (weighted CE then focal), and the three-stage concept curriculum.

layers. The encoder is left untouched; the systems differ only in what sits on top of it and in how it is trained.

The training configuration is fixed across systems (Sect. 5). The seed is 42 and all runs shared device and library versions; the tokenizer, label mapping and split are identical everywhere (bitwise determinism across hardware is not claimed). Figure 1 overviews the pipeline and the points at which the six systems differ.

4.2 Systems and the Ablation Chain

The systems form a chain in which each step swaps a whole recipe of decisions (head, losses, learning rates, stages and decision rule together), plus two probes that target a single component. We label them **M1**–**M6** and use these labels throughout:

1. **M1**, SENSITIVEAI-VI (baseline): standard Hugging Face head, single rate 3×10^{-5}, unweighted cross-entropy, argmax.

2. **M2**, SENSITIVEAI-VI-CUSTOM: baseline plus the multi-view head (Sect. 4.3).
3. **M3**, SENSITIVEAI-VI-CUSTOMDLR2STAGE: custom head plus discriminative learning rates, two-stage training, class-weighted cross-entropy then focal loss, and searched thresholds (Sect. 4.5).
4. **M4**, SENSITIVEAI-VI-CUSTOM-CURRICULUM: custom head plus a three-stage "concept" curriculum instead of the two stages (Sect. 4.6).
5. **M5**, SENSITIVEAI-VI-CUSTOMDLR2STAGE-EXT: probe continuing M3's stage 1 for 15 further epochs, to test whether that optimum was a peak or merely the end of the schedule.
6. **M6**, SENSITIVEAI-VI-STAGE2: probe applying *only* the stage-2 recipe (focal loss, class weights, lower rate) on the baseline checkpoint, keeping the standard head.

4.3 Multi-View Classification Head

The standard DeBERTa head places a pooler on the first token and a single linear layer on top, $\mathbf{h}[0] \to \text{Linear}(768{\to}768) + \tanh \to \text{Linear}(768{\to}3)$, so the decision rests on one token—fragile for short, noisy comments where a trigger word, not the first token, decides the label. The custom head keeps the encoder and changes only what follows. With final states $\mathbf{H} \in \mathbb{R}^{L\times 768}$ and mask $\mathbf{m} \in \{0,1\}^L$, the three views are $\mathbf{z}_{\text{cls}} = \mathbf{H}[0]$, $\mathbf{z}_{\text{mean}} = \frac{\sum_l m_l \mathbf{H}[l]}{\sum_l m_l}$ and $\mathbf{z}_{\max} = \max_{l:\, m_l=1} \mathbf{H}[l]$, where the masked mean ignores padding and the masked max maximises over valid positions only. The views are concatenated into a 2304-dimensional vector, LayerNorm-normalised, and passed through a fusion block $\text{Linear}(2304{\to}512) \to \text{GELU} \to \text{Dropout}(0.1) \to \text{LayerNorm} \to \text{Dropout}(0.1) \to \text{Linear}(512{\to}3)$; the 512-unit bottleneck adds a non-linear, regularised boundary. Since the encoder, data and optimiser are unchanged, M1 vs. M2 is the one clean single-factor step in the chain and isolates the head.

4.4 Losses and Class Weighting

The baseline uses unweighted cross-entropy; later systems use *weighted cross-entropy* (class weights = inverse square root of the training frequency, normalised to mean one), *focal loss* [9] with $\gamma = 2$ and $\alpha_{\text{HATE}} = 0.5$, $\alpha_{\text{OFFENSIVE}} = \alpha_{\text{CLEAN}} = 0.25$ combined with those weights, or *manual weights* HATE:3, OFFENSIVE:2, CLEAN:1 for the curriculum's final stage. Class weights are computed on the training split only.

4.5 Discriminative Learning Rates and Two-Stage Training

Two optimisation ideas are combined in M3.

Discriminative (layer-wise) learning rates. Following ULMFiT [8], parameters are split into four groups with decreasing rates from head to input (embeddings 5×10^{-6}, lower half 1×10^{-5}, upper half 2×10^{-5}, head 1×10^{-4}), so the

task-specific head adapts quickly from random initialisation while pre-trained representations avoid catastrophic forgetting. The encoder starts from the same checkpoint as M1 and M2, so the recipes share one initialisation.

Two-stage training. Training has two stages of up to 15 epochs each: *stage 1* stabilises the representation under class-weighted cross-entropy at rate 2×10^{-5} with warmup 0.1; *stage 2* continues from its checkpoint with lower rates and focal loss, which down-weights easy examples. Here the selected checkpoint was the *end of stage 1* (validation macro-F1 0.7089); focal stage 2 never improved on it and stopped early, so M3's headline number comes from discriminative rates plus class-weighted cross-entropy, not focal loss.

Label-specific decision thresholds. The decision rule is changed from argmax to per-class thresholds: a label is accepted when its probability reaches its own threshold, the highest-probability accepted label wins, and if none passes the plain argmax is used. The thresholds are searched to maximise validation macro-F1 by coordinate ascent over a 101-point grid in $[0, 1]$ from 0.5, on the validation split only, then frozen and applied unchanged to the test pass.

4.6 Three-Stage Curriculum Concept Training

M4 replaces the two stages with a three-stage curriculum [1] that shapes the representation with easier binary sub-problems before the final decision: *stage 1* targets "OFFENSIVE versus anything else", *stage 2* "HATE versus anything else", and *stage 3* returns to the three-class objective with class weights HATE:3, OFFENSIVE:2, CLEAN:1. The full dataset is used in *every* stage (a concept's negative class is the other two classes) and the three-way head is kept throughout, the concept loss simply re-reading the three logits as a binary objective. Each stage runs up to 10 epochs with rates decreasing from 2×10^{-5} to 1×10^{-5} (stage 3 raises this slightly to 1.5×10^{-5}) and the discriminative grouping of Sect. 4.5. M4 asks whether a finer sequence of increasingly difficult stages beats M3's two. All three stages run, but no epoch after the first improves the validation macro-F1 used for checkpoint selection (best 0.694), so the finer sequence does not win.

5 Experimental Setup

5.1 Hardware and Software

All experiments were run on a single workstation with an NVIDIA GeForce RTX 4060 Laptop GPU (8 GB), CUDA 12.6, PyTorch 2.13 and Python 3.11, using the Hugging Face Transformers library [22]. The `mdeberta-v3-base` encoder has 278.2M parameters. The standard Hugging Face head (a 768-d pooler plus a 3-way linear layer) adds 0.59M, so the standard-head systems (M1, M6) total 278.8M; the multi-view head adds 1.19M, so the custom-head systems (M2–M5) total 279.4M. The largest gap between any two systems is thus +0.21% of the parameters, and we treat them as of essentially identical size.

Table 3. Training configuration and the validation checkpoint that was frozen for the test pass (M1–M6, Sect. 4.2). AdamW, linear schedule, effective batch size 32 and sequence length 256 are shared by all systems. "init" is the encoder the first stage starts from; "best ep." is the epoch whose validation macro-F1 was highest (it can lie in an earlier stage).

System	init	stages	loss	class w.	best ep.	val M-F1
M1	pre-trained	1	CE	none	9	0.6961
M2	pre-trained	1	CE	none	14	0.6957
M3	chained	2	CE $\rightarrow$ focal	inv-sqrt	15 (s1)	0.7089
M4	M3 best	3	concept $\rightarrow$ CE	manual	1 (s1)	0.6942
M5	M3 stage-1	1	CE	inv-sqrt	3	0.7072
M6	M1 best	1	focal	inv-sqrt	4	0.6928

5.2 Training Protocol

All systems share one configuration: AdamW [10] with weight decay 0.01, a linear schedule with warmup 0.05–0.1, batch size 16 with gradient accumulation 2 (effective 32), gradient clipping at norm 1.0, sequence length 256 tokens, bf16 training (fp32 for the latency benchmark), and up to 15–30 epochs depending on the recipe. Early stopping monitors validation macro-F1 with patience 6 and restores the best checkpoint. The systems differ only in the encoder initialisation, the number of stages, the loss sequence and the class-weighting scheme (Sect. 4.2); the test split is evaluated exactly once per system with the checkpoint selected on validation. Table 3 lists the resolved configuration and the frozen checkpoint (best epoch and its validation macro-F1) for each system. Every system reads the same three split files (verified by SHA-256 digest) and label mapping, and all class weights and thresholds are estimated on training and validation only, never on test; only the head, learning rates, number of stages and loss vary.

5.3 Metrics

Because of the imbalance described in Sect. 3.2, we treat *macro-averaged F1* as primary and report accuracy only as a diagnostic, alongside per-class precision, recall and F1 and macro ROC-AUC / average precision (AP), since precision-recall summaries are more trustworthy than ROC under imbalance [18]. Calibration uses the top-label expected calibration error (ECE, 10 bins); significance uses a percentile bootstrap (macro-F1) and the exact McNemar test (accuracy).

6 Results

6.1 Headline Comparison

Table 4 reports the test results. The systems are tightly clustered—macro-F1 spans 0.6536–0.6638 (1.0 pt) and accuracy 0.8633–0.8783—and these spreads sit inside sampling noise: the 95% bootstrap intervals on macro-F1 are about

Table 4. Comparison of the six systems (M1–M6, Sect. 4.2) on the ViHSD test set under one fixed split; M-F1/W-F1: macro/weighted F1; HATE/OFF./CLEAN: per-class F1; AUC/AP: macro ROC-AUC / average precision.

Model	Acc.	M-F1	W-F1	HATE	OFF.	CLEAN	AUC	AP
M1	0.8734	0.6545	0.8705	0.5864	0.4367	0.9405	0.8873	**0.6599**
M2	0.8762	0.6575	0.8709	0.5830	0.4493	0.9403	0.8842	0.6484
M3	0.8754	**0.6638**	**0.8730**	0.5943	**0.4561**	0.9409	0.8594	0.6254
M4	0.8731	0.6590	0.8701	0.5847	0.4535	0.9389	0.8500	0.6367
M5	**0.8783**	0.6536	0.8711	0.5843	0.4350	**0.9415**	0.8096	0.6124
M6	0.8633	0.6604	0.8679	**0.6088**	0.4382	0.9344	**0.8930**	0.6534

± 0.019 wide (Table 5), so every pairwise gap lies within a single interval and none is significant (Sect. 6.6). The observations below are directions of effect, not established rankings.

The full recipe (M3) is best by macro-F1 (**0.6638**) and OFFENSIVE F1 (0.4561); the extended probe (M5) has the highest accuracy (**0.8783**) and CLEAN F1 yet the lowest macro ROC-AUC (0.8096); the stage-2 probe (M6) has the highest HATE F1 (**0.6088**) and macro ROC-AUC (**0.8930**) but the lowest accuracy, while the baseline (M1) holds the highest macro average precision (AP, 0.6599, ahead of M6's 0.6534); and the custom head alone moves macro-F1 only to 0.6575.

6.2 Ablation Reading

Table 4 gives the chain: each step swaps a *whole* recipe, so a gap is the combined effect of its changes, not a single-factor estimate. The custom head adds +0.30 macro-F1 (M1→M2); discriminative rates, two-stage class-weighted training and a threshold search another +0.63 (M2→M3); and the curriculum gives back 0.48 (M3→M4), but all three stages ran and the frozen M4 checkpoint is still stage 1, epoch 1 (val. macro-F1 0.6942): the later stages never won validation selection. The two probes are most informative: class-weighted focal loss on a *standard* head (M6) reaches 0.6604, beating the baseline, the custom head and the curriculum at the best HATE F1 and macro ROC-AUC (though not macro AP, where M1 leads); extending stage 1 (M5) *hurts* (macro-F1 0.6536, ROC-AUC 0.8096), evidence that further cross-entropy training drifts to the majority class. M3's selected checkpoint was the end of its cross-entropy stage (val. macro-F1 0.7089 at epoch 15), so its focal stage never helped: the same ingredient succeeds or fails depending on what precedes it.

6.3 Accuracy Versus Macro-F1

The clearest lesson is the accuracy/macro-F1 contrast: the most accurate system (M5, 0.8783) is the one *not* to deploy—it has the worst OFFENSIVE recall (0.381) and the weakest ranking quality, because majority-class prediction is rewarded by accuracy and punished by macro-F1. Accuracy-based selection is

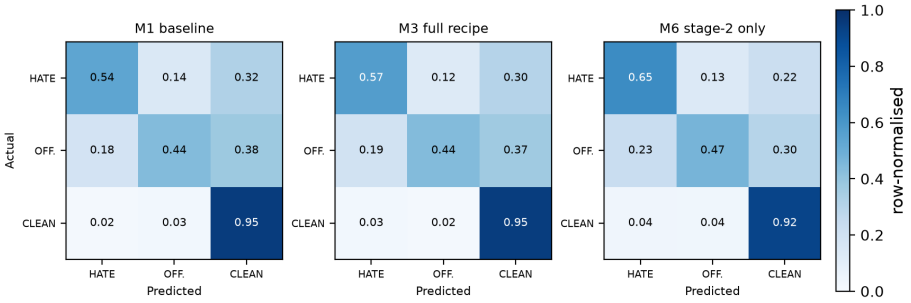


Fig. 2. Row-normalised test confusion matrices for the baseline (M1), the best macro-F1 system (M3) and the focal-only probe (M6). Every system shares one failure mode: OFFENSIVE comments sent to HATE or CLEAN, and HATE diluted into its neighbours; the systems differ by a few percent along these rows, not by a change of failure mode.

therefore inadvisable here, consistent with the classical preference for precision-recall over ROC under imbalance [18].

6.4 Training Dynamics

Validation accuracy plateaus within about five epochs in a narrow 0.86–0.88 band, but the selected checkpoints fall at different epochs (1 for M4, 3 for M5, 4 for M6, 9 for M1, 15 for M3) and validation loss turns upward while training loss keeps falling, so early stopping on validation macro-F1—not loss—selects the checkpoint.

6.5 Per-Class Behaviour and Error Analysis

The per-class breakdown (Table 4) localises the difficulty: CLEAN is nearly solved (F1 0.93–0.94) while OFFENSIVE stays at 0.43–0.46 and HATE at 0.58–0.61 for every system. The full recipe rebalances the minority classes—HATE recall 0.538 → 0.573 and OFFENSIVE precision 0.434 → 0.474 at unchanged recall—while the focal-only probe (M6) is the most minority-heavy system of the six: it has the highest HATE recall (0.647) and OFFENSIVE recall (0.471), bought with the lowest CLEAN recall (0.922), which is why it loses the most accuracy on an 83%-CLEAN test set.

Figure 2 confirms the shared failure mode: the OFFENSIVE row is weakest (38–46% misrouted) and HATE is diluted into its neighbours, while CLEAN is almost never touched. Splitting the test set by surface cues (M3) localises the difficulty: macro-F1 is lowest on *very long* comments (0.61; accuracy 0.786, OFFENSIVE recall 0.14) and on *teencode* or *no-diacritic* comments, highest on short ones—descriptive gaps on one split, not a causal claim.

6.6 Statistical Significance and Calibration

Because the spread is so small we re-analyse the frozen predictions with a percentile bootstrap (10,000 resamples, seed 42) for macro-F1 and the exact two-sided McNemar test for correctness; the two tests answer different questions

Table 5. Post-hoc statistics on the frozen test predictions. *Top:* accuracy and macro-F1 with 95% percentile bootstrap intervals (10,000 resamples of the 6,680 test items) and the top-label expected calibration error (ECE, 10 bins). *Bottom:* paired comparisons, A minus B, with a 95% paired-bootstrap interval on macro-F1; p_{F1} is the paired-bootstrap p-value and p_{acc} the exact two-sided McNemar p-value on the two systems' correctness. † marks $p < 0.05$ after a Holm–Bonferroni correction over the seven comparisons below; values below 0.001 are shown as < 0.001. No macro-F1 gap is significant; the only differences that survive correction are the two accuracy gaps involving M6, discussed in the text.

System	Acc. [95% CI]	M-F1 [95% CI]	ECE	
M1	0.8734 [0.8653, 0.8813]	0.6545 [0.6353, 0.6736]	0.106	
M2	0.8762 [0.8681, 0.8841]	0.6575 [0.6375, 0.6771]	0.111	
M3	0.8754 [0.8672, 0.8835]	0.6638 [0.6442, 0.6823]	0.120	
M4	0.8731 [0.8648, 0.8810]	0.6590 [0.6396, 0.6779]	0.119	
M5	0.8783 [0.8702, 0.8861]	0.6536 [0.6334, 0.6728]	0.118	
M6	0.8633 [0.8549, 0.8716]	0.6604 [0.6424, 0.6781]	0.068	
$A - B$	ΔM-F1 [95% CI]	p_{F1}	ΔAcc	p_{acc}
M3 − M1	+0.0092 [-0.0082, +0.0269]	0.298	+0.0021	0.526
M3 − M2	+0.0062 [-0.0060, +0.0186]	0.328	-0.0007	0.788
M3 − M4	+0.0047 [-0.0051, +0.0144]	0.342	+0.0024	0.217
M3 − M6	+0.0033 [-0.0124, +0.0188]	0.678	+0.0121	$< 0.001^{\dagger}$
M3 − M5	+0.0101 [-0.0023, +0.0228]	0.113	-0.0028	0.217
M5 − M1	-0.0009 [-0.0185, +0.0164]	0.906	+0.0049	0.110
M6 − M1	+0.0059 [-0.0071, +0.0191]	0.377	-0.0100	$< 0.001^{\dagger}$

and are reported separately. Table 5 gives the intervals, the top-label expected calibration error (ECE) and the paired comparisons.

Two conclusions follow. First, **no pairwise macro-F1 difference is significant**: the largest gap (M3−M5, +0.010) has paired interval $[-0.002, +0.023]$ containing zero ($p = 0.11$). Second, **the accuracy ranking is more stable, and there the McNemar test does reject**: M6 is significantly *less* accurate than both M3 ($p = 1.6 \times 10^{-4}$) and M1 ($p = 2.7 \times 10^{-4}$)—the only two of the seven comparisons in Table 5 that survive a Holm–Bonferroni correction—because M6 buys the highest minority recall by giving up CLEAN recall (0.922), and on a test set that is 83% CLEAN that exchange costs accuracy. Calibration also differs (focal-only probe best, ECE 0.068; full recipe worst, 0.120).

Non-rejection is not equivalence: one seed and 6,680 test items support only "no macro-F1 difference detected"; proving equivalence would need a pre-specified margin and a powered test over several seeds. We therefore claim neither "M3 is best" nor "the recipes are equal", only that the choice should rest on secondary criteria (minority recall, calibration, cost).

6.7 Ranking Quality and Decision Rules

Ranking tells a different story from thresholded F1: macro ROC-AUC peaks for M6 (0.8930) and M1 (0.8873) while M3 sits at 0.8594, consistent with their dif-

Table 6. Published macro-F1 on ViHSD vs. our best system (reported values, not re-trained by us). External rows differ in split handling, preprocessing, encoder size and model selection, none of which we control; SMTCE rows are as tabulated in [16].

System	Training setting	Macro-F1
Dataset baseline [11]	reported baseline	62.69
XLM-R [16,2]	multilingual encoder	63.68
mBERT (cased) [16]	multilingual encoder	64.20
PhoBERT [16,13]	monolingual encoder	66.07
viBERT4news [16]	monolingual encoder	66.43
Ours (best system, M3)	mDeBERTa-v3 + full recipe	**66.38**
PhoBERT-CNN [20]	+ augmentation / preprocessing	67.46
ViSoBERT [15]	social-media encoder	68.77

fering calibration. Macro average precision (AP) is less kind to M6: it peaks for M1 (0.6599, M6 second at 0.6534), so the two ranking summaries disagree and no single ranking number crowns M6. Only M3–M6 searched per-class thresholds, and only M4 found non-trivial values (0.01/0.5/0.64, validation macro-F1 0.694 $\rightarrow$ 0.697)—a gain that did not transfer to test; elsewhere the search returned the trivial 0.5, so a searched rule adds nothing over calibrated argmax here.

6.8 Inference Cost

Latency (fp32, same GPU/tokenizer/length/batch, indicative only) clusters at 11.0–11.6 ms/sample for the custom-head systems, but the standard-head baseline (M1) measured 28.1 ms against 10.7 ms for the architecturally identical probe (M6), so no throughput claim is made (Sect. 7.2).

6.9 External Comparison

Table 6 compares with published ViHSD results. Our best macro-F1 (0.6638) is comparable to the strongest published monolingual encoders (PhoBERT 66.07) and above the multilingual ones, but below recent Vietnamese-specific encoders (viBERT4news 66.43, ViSoBERT 68.77) and augmentation-based systems; the rows differ in split handling, preprocessing and model selection and none is retrained by us, so the table is indicative.

7 Discussion

7.1 Which Decisions Actually Matter

All recipes sit within 1.0 macro-F1 point and, as Sect. 6.6 shows, these gaps are not statistically distinguishable under our tests (not proof of equivalence), so the ablation gives only the direction of the effects. Class-weighted focal loss on a plain head (M6) reached 0.6604, matching the full recipe within noise while

giving the best minority recall and macro ROC-AUC (its macro AP is second to M1's), so the loss change gave the largest observed shift; the multi-view head alone adds only +0.30 macro-F1. More stages did not help: both multi-stage systems were frozen at a stage-1 checkpoint, so the extra stages never improved validation selection. The threshold search collapsed to 0.5 (M3, M5, M6) or failed to transfer (M4), so the decision rule added no gain.

The accuracy leader is also the macro-F1 laggard (Sect. 6.3), a direct consequence of an 83%-majority test set: each extra CLEAN prediction raises accuracy even when it misclassifies a minority comment, so an accuracy-optimised system would under-report offensive and hateful content—the opposite of the product requirement. Selection should therefore use macro-F1 (or minority recall), driven by validation so the test split stays honest.

7.2 Threats to Validity

Each system was trained once (seed 42), so the estimates carry run-to-run variance the shared split and seed do not remove; the bootstrap intervals quantify sampling noise but not training noise, and as the observed gaps already lie inside one interval we claim no recipe is reliably better. The official split reuses comments across train/dev/test (Sect. 3); we report this rather than hide it, and the de-duplicated check leaves the macro-F1 ordering effectively unchanged (Table 2), but absolute scores are optimistic under a strictly leakage-free protocol. ViHSD is the only corpus, its external baselines (Table 6) reprint published numbers under other settings, and the latency runs were inconsistent between identical architectures, so no cost claim is made. Agreement is only moderate ($\kappa \approx 0.5$), so part of the error is irreducible.

7.3 Implications

For applied moderation the message is negative but worth reporting: no single trick solves Vietnamese hate speech detection, and added complexity—curricula, searched thresholds, longer training—did not pay off. The minority classes remain hard (OFFENSIVE F1 stuck near 0.46 across all six systems), which is consistent with the labelled data, not the optimiser, being the binding constraint; more labelled offensive examples or span-level supervision as in ViHOS [7] are plausible next steps.

8 Conclusion

We presented SensitiveAI, a controlled comparison of six optimisation recipes for three-class Vietnamese hate speech detection on UIT-ViHSD. Holding the split, backbone, tokenizer and seed fixed while swapping *whole* recipes—head, losses, rates, stages and decision rule together—lets us read the chain as the combined direction of each design choice. Our main—deliberately modest—finding is that we *detected no* macro-F1 difference among the recipes: the 1.0-point spread lies

within one paired-bootstrap interval and no pairwise difference is significant. This is a failure to detect, not proof of equivalence. The only robust paired differences are in *accuracy*: the focal-only probe (M6), which reaches the highest minority recall by giving up CLEAN recall, is significantly less accurate than M3 and M1 (McNemar $p < 0.001$, both surviving a Holm–Bonferroni correction over the seven reported comparisons), so the highest-accuracy system (M5) is also the macro-F1 laggard—accuracy is unsafe on an 83%-majority corpus. The best macro-F1 (0.6638, M3) edges the baseline by under a point; M6 gives the best HATE F1 (0.6088) and macro ROC-AUC (0.8930) at a slightly lower macro-F1 (0.6604); the custom head alone adds only +0.30. Multi-stage training never won on validation—both multi-stage systems froze a stage-1 checkpoint, so the three-stage curriculum added cost but no selected gain. OFFENSIVE F1 stays near 0.46 for every recipe, consistent with the labelled data, not the optimiser, being the binding constraint.

Future Work. Repeat each system with several seeds to quantify *training* variance and test the gaps against it; re-train the published baselines under the same split so Table 6 becomes a controlled comparison; and enrich the data (span-level supervision, hard-negative retrieval, targeted OFFENSIVE examples). A controlled latency study is still needed before any throughput claim about the custom head.

Acknowledgements. All experiments were run on a single consumer GPU (NVIDIA GeForce RTX 4060 Laptop, 8 GB). The authors thank the creators and annotators of the UIT-ViHSD dataset and the maintainers of the PyTorch and Hugging Face Transformers libraries, on which this study is built.

Data and Code Availability. The code, the six configuration files, the split manifest with SHA-256 digests (and the script that rebuilds the identical split), the frozen per-model test predictions, the post-hoc analysis reports and the scripts that reproduce every table and figure are available at `https://anonymous.4open.science/r/AIModerationContent-ECEE/`.

References

1. Bengio, Y., Louradour, J., Collobert, R., Weston, J.: Curriculum learning. In: Proceedings of the 26th International Conference on Machine Learning (ICML). pp. 41–48 (2009)
2. Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., Stoyanov, V.: Unsupervised cross-lingual representation learning at scale. Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL) pp. 8440–8451 (2020)
3. Cui, Y., Jia, M., Lin, T.Y., Yang, J., Belongie, S.: Class-balanced loss based on effective number of samples. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 9268–9277 (2019)
4. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of NAACL-HLT. pp. 4171–4186 (2019)
5. He, P., Gao, J., Chen, W.: DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In: International Conference on Learning Representations (ICLR) (2023)
6. He, P., Liu, X., Gao, J., Chen, W.: DeBERTa: Decoding-enhanced BERT with disentangled attention. In: International Conference on Learning Representations (ICLR) (2021)

7. Hoang, P.G., Luu, C.D., Tran, K.Q., Nguyen, K.V., Nguyen, N.L.T.: ViHOS: Hate speech spans detection for vietnamese. In: Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL). pp. 652–669 (2023)
8. Howard, J., Ruder, S.: Universal language model fine-tuning for text classification. In: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL). pp. 328–339 (2018)
9. Lin, T.Y., Goyal, P., Girshick, R., He, K., Dollár, P.: Focal loss for dense object detection. In: Proceedings of the IEEE International Conference on Computer Vision (ICCV). pp. 2980–2988 (2017)
10. Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: International Conference on Learning Representations (ICLR) (2019)
11. Luu, S.T., Nguyen, K.V., Nguyen, N.L.T.: A large-scale dataset for hate speech detection on vietnamese social media texts. In: Advances and Trends in Artificial Intelligence. Artificial Intelligence Practices (IEA/AIE). pp. 415–426. Lecture Notes in Computer Science, Springer (2021)
12. Microsoft: mDeBERTa-v3: Multilingual DeBERTa-v3 pre-trained language model. Hugging Face model card (2021)
13. Nguyen, D.Q., Nguyen, A.T.: PhoBERT: Pre-trained language models for vietnamese. In: Findings of the Association for Computational Linguistics: EMNLP 2020. pp. 1037–1042 (2020)
14. Nguyen, L.T., Nguyen, K.V., Nguyen, N.L.T.: Constructive and toxic speech detection for open-domain social media comments in vietnamese. In: Advances and Trends in Artificial Intelligence. Artificial Intelligence Practices (IEA/AIE). pp. 572–583. Springer (2021)
15. Nguyen, N., Phan, T., Nguyen, D.V., Nguyen, K.V.: ViSoBERT: A pre-trained language model for vietnamese social media text processing. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP) (2023)
16. Nguyen, N.L.T., et al.: SMTCE: A social media text classification evaluation benchmark and BERTology models for vietnamese. In: Proceedings of the 36th Pacific Asia Conference on Language, Information and Computation (PACLIC) (2022)
17. Nguyen, T.T., Nguyen, H., Pham, D.V., Nguyen, C.V., Do, T.H., Nguyen, K.V.: ViHateT5: A T5-based model for vietnamese hate speech detection. In: Findings of the Association for Computational Linguistics: ACL 2024 (2024)
18. Saito, T., Rehmsmeier, M.: The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets. PLOS ONE **10**(3), e0118432 (2015)
19. Tran, C.D., Nguyen, H., Pham, Q., Nguyen, V.H., Nguyen, P., Luu, A.T., Nguyen, H.T.H., Vu, H., Le, H., Nguyen, D.T.N., et al.: ViDeBERTa: A powerful pre-trained language model for vietnamese. In: Findings of the Association for Computational Linguistics: EACL 2023. pp. 1041–1048 (2023)
20. Tran, K.Q., Nguyen, A.T., Hoang, P.G., Luu, C.D., Do, T.H., Nguyen, K.V.: Vietnamese hate and offensive detection using PhoBERT-CNN and social media streaming data. arXiv preprint arXiv:2206.00524 (2022)
21. Vu, X.S., Vu, T., Tran, M., Nguyen, L.M.: Vietnamese hate speech detection: VLSP shared task. In: Proceedings of the 7th International Workshop on Vietnamese Language and Speech Processing (VLSP) (2020)
22. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., et al.: Transformers: State-of-the-art natural language processing. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations. pp. 38–45 (2020)